

Your Loan & Investment Calculator: A Beginner's Guide

Written for Yamen Agha. Read it slowly, with the app and the Excel file open next to you.

This guide starts from zero. You know from your accounting courses how interest is **recorded**: interest expense, accrued interest, the split of a loan into current and non-current portions. Here you learn how interest is **calculated**, and how one simple idea, the *time value of money*, explains loans, savings plans and investment decisions. By the end you should be able to:

1. Explain the time value of money, compounding, and the difference between APR and EAR.
2. Build an amortization schedule by hand and explain why early payments are mostly interest.
3. Explain what extra payments do, and compare annuity, equal-principal and interest-only loans.
4. Project an investment with contributions, inflation and tax, and explain NPV and IRR in plain words.
5. Build a loan schedule and an investment table in Excel from an empty sheet.
6. Present the project to an employer, answer interview questions, and honestly say "I understand this and I improved it myself."

Every number in this guide was produced by the app's engine (`finance_calc.py`). The key figures were also checked against LibreOffice's Excel functions.

Contents

1. Time value of money, from zero
2. Interest and compounding
3. APR vs EAR, and the rate per period
4. How an amortization schedule works (worked by hand)
5. Why early payments are mostly interest
6. Extra payments
7. Loan types
8. Investing: compound growth with contributions
9. Inflation, real returns and tax
10. NPV and IRR in plain words
11. How to use the app and the Excel file
12. Hands-on Excel exercise: build it yourself
13. Presenting the project to an employer
14. Interview questions and sample answers
15. Your study plan for the next 7 days

1. Time value of money, from zero

A dollar today is worth more than a dollar next year. There are three reasons:

- you can invest today's dollar and earn interest on it
- prices tend to rise (inflation), so a future dollar buys less

- a promised future dollar might never arrive (risk)

So money at different dates can't simply be added together. First you move it to the **same date**, using an interest rate.

Moving money forward: future value (FV)

If you invest \$100 at 5% a year:

- after 1 year: $100 \times 1.05 = \mathbf{\$105.00}$
- after 10 years: $100 \times 1.05^{10} = \mathbf{\$162.89}$

$$\mathbf{FV = PV \times (1 + r)^n}$$

Moving money back: present value (PV)

How much is \$1,000 received in 5 years worth today, if you could earn 8% a year? Divide instead of multiplying:

$$\mathbf{PV = FV / (1 + r)^n = 1,000 / 1.08^5 = \$680.58}$$

This is called **discounting**, and r is the **discount rate**. Everything else in this guide, from loans to savings plans to NPV, is just this idea applied many times.

The five TVM variables

Every TVM problem has five quantities. If you know four, Excel can find the fifth:

Variable	Meaning	Excel function to solve for it
RATE	Interest rate per period	<code>RATE(nper, pmt, pv, [fv], [type])</code>
NPER	Number of periods	<code>NPER(rate, pmt, pv, [fv], [type])</code>
PMT	Payment each period	<code>PMT(rate, nper, pv, [fv], [type])</code>
PV	Value today	<code>PV(rate, nper, pmt, [fv], [type])</code>
FV	Value at the end	<code>FV(rate, nper, pmt, [pv], [type])</code>

Excel's sign convention: money you *receive* is positive, and money you *pay out* is negative. That is why `=PMT(0.5%, 360, 200000)` gives **−1,199.10**: you receive \$200,000 and pay back \$1,199.10 a month. `type` is 0 for payments at the **end** of each period and 1 for payments at the **beginning**.

2. Interest and compounding

Simple interest is paid on the original amount only. **Compound interest** is also paid on the interest you have already earned. For \$100 at 10% for 3 years:

Year	Simple interest	Compound interest
1	110.00	110.00
2	120.00	121.00
3	130.00	133.10

The gap looks small, but over decades it becomes most of the money (see section 8).

The rule of 72 is a quick mental check: money doubles in roughly $72 / (\text{rate in \%})$ years. At 7%, $72 / 7 \approx 10.3$ years. The exact answer is $\ln 2 / \ln 1.07 = 10.24$ years.

Compounding frequency is how often interest is added to the balance: yearly, monthly, daily, or in the limit, continuously. The more often it compounds, the more you end up with at the same quoted rate. That leads to the next section.

3. APR vs EAR, and the rate per period

Banks quote a **nominal annual rate (APR)**, for example "12% compounded monthly". That really means $12\% / 12 = 1\% \text{ per month}$. Because each month's interest earns interest, the true yearly growth is a little more than 12%:

$$\text{EAR} = (1 + \text{APR}/m)^m - 1 = 1.01^{12} - 1 = 12.68\%$$

Here m is the number of compoundings a year. The **EAR** (effective annual rate, called **APY** for savings) is the one number you can compare across products. In Excel, `=EFFECT(12%, 12)` gives 12.68%, and `=NOMINAL(12.68%, 12)` goes back to 12%.

The app's APR → EAR table, for 6% APR:

Compounding	Annual	Semi-annual	Quarterly	Monthly	Bi-weekly	Weekly	Daily	Continuous
EAR	6.0000%	6.0900%	6.1364%	6.1678%	6.1763%	6.1800%	6.1831%	6.1837%

Continuous compounding is the upper limit: $e^{0.06} - 1$.

When compounding and payment frequencies differ

Usually they are the same: a monthly loan compounds monthly, so the rate per payment is simply **APR / 12**. Sometimes they differ. Canadian mortgages, for example, compound **semi-annually** but are paid **monthly**. Dividing 6% by 12 would then be wrong. Instead, the app uses the **equivalent periodic rate**: the monthly rate that gives the same EAR.

$$i = (1 + \text{APR}/m)^{(m/p)} - 1$$

Here m is compoundings a year and p is payments a year. For 6% compounded semi-annually and paid monthly: $i = 1.03^{(1/6)} - 1 = 0.49386\%$ a month, instead of 0.5%. On a \$200,000, 30-year loan, the payment is **\$1,189.65** instead of \$1,199.10.

The investment mode works the same way. If the return compounds monthly but you contribute quarterly, the rate per quarter is $1.005833...^3 - 1$. The app prints the rate it used under the inputs.

4. How an amortization schedule works (worked by hand)

An **amortizing loan** is repaid in equal payments. Each payment covers that period's interest first, and the rest repays principal. The equal payment comes from the annuity formula:

$$PMT = P \times i / (1 - (1 + i)^{-n})$$

A small example you can do with a calculator

Borrow **\$10,000 at 12% APR for 12 months**. Then $i = 1\%$ per month and $n = 12$.

$PMT = 10,000 \times 0.01 / (1 - 1.01^{-12}) = 10,000 \times 0.01 / 0.112551 = \mathbf{\$888.49}$. In Excel: `=PMT(1%, 12, -10000)`.

Every row follows the same four steps:

1. **Interest** = opening balance $\times 1\%$, rounded to the cent.
2. **Principal** = payment – interest.
3. **Closing balance** = opening balance – principal.
4. The closing balance becomes next month's opening balance.

#	Opening balance	Payment	Interest (1%)	Principal	Closing balance
1	10,000.00	888.49	100.00	788.49	9,211.51
2	9,211.51	888.49	92.12	796.37	8,415.14
3	8,415.14	888.49	84.15	804.34	7,610.80
...	...	...	...	...	...
11	1,750.65	888.49	17.51	870.98	879.67
12	879.67	888.47	8.80	879.67	0.00

Try rows 1–3 yourself before reading on. For row 2: $9,211.51 \times 1\% = 92.1151$, which rounds to 92.12. Then $888.49 - 92.12 = 796.37$, and $9,211.51 - 796.37 = 8,415.14$.

The total interest is **\$661.86**. Notice two things:

- **The last payment is \$888.47, not \$888.49.** The payment was rounded to the cent, so after 11 payments there is a tiny difference. A correct schedule makes the **last payment absorb it**: last payment = remaining balance + its interest. Then the balance ends at exactly 0.00. The first version of the Excel export missed this, and its balance never reached zero.
- **Interest falls and principal rises every month**, because interest is charged on a shrinking balance.

Excel's `IPMT` and `PPMT` give the interest and principal parts of any one payment: `=IPMT(1%, 2, 12, -10000)` = 92.12 (rounded). They use the *unrounded* payment, so in the last row `PPMT` gives 879.69

while the rounded schedule shows 879.67. That two-cent gap is exactly the rounding that the final payment cleans up.

5. Why early payments are mostly interest

Take the app's default loan: **\$200,000 at 6% for 30 years, paid monthly** ($i = 0.5\%$, $n = 360$).

- Payment: **\$1,199.10**. Total interest over 30 years: **\$231,677.04**, more than the loan itself.
- Payment 1: interest = $200,000 \times 0.5\% = \mathbf{\$1,000.00}$, so only **\$199.10** goes to principal.
- Year 1: interest **\$11,933.19**, principal \$2,456.01. Excel agrees: `=CUMIPMT(0.5%, 360, 200000, 1, 12, 0)` = -11,933.19.
- The first 5 years: you pay \$71,946.00, of which **\$58,054.80 is interest** and only \$13,891.20 is principal.
- After 10 years you still owe **\$167,371.60**.
- Principal is larger than interest for the first time at **payment 223** (May 2045), more than 18 years in.

The reason: interest is charged on the **balance**, and early on the balance is huge. As the balance falls, the interest part falls, and more of the fixed payment goes to principal. The "Payment Composition" chart in the app shows this as orange (interest) shrinking and blue (principal) growing.

The rate matters a lot. The same \$200,000 over 30 years costs \$954.83 a month at 4%, \$1,199.10 at 6% and \$1,467.53 at 8%. Choosing a **15-year** term at 6% raises the payment to \$1,687.71 but cuts total interest to **\$103,788.82**.

6. Extra payments

An extra payment goes **straight to principal**. A lower balance means less interest next month, so more of every later payment goes to principal. The effect snowballs. The regular payment stays the same, so the loan simply **ends sooner**.

On the \$200,000 loan:

Extra each month	Payments	Paid off	Interest saved
\$0	360	Oct 2056	—
\$100	295	May 2051	\$49,138.85
\$200	252	Oct 2047	\$79,800.86
\$500	179	Sep 2041	\$129,142.55

With **\$200 a month extra**, interest drops from \$231,677.04 to **\$151,876.18**. That is 108 fewer payments, **9 years earlier**. You pay \$50,200 extra in total and save \$79,800.86 of interest. The last payment is a small \$702.08. A one-off **\$10,000 lump sum at payment 12** saves \$41,044.36 and removes 42 payments.

Check with NPER: `=NPER(0.5%, -1399.10, 200000)` = 251.5, so you need 252 payments. That matches the schedule.

Two caveats for real life. Some loans charge **prepayment penalties**. And paying extra on a 6% loan is like earning a guaranteed 6%, which you should compare with what else you could do with the money.

7. Loan types

All three below are \$200,000 at 6% over 30 years:

Type	How it works	Payment	Total interest
Annuity (fixed payment)	The same payment every month, from <code>PMT</code> . Most mortgages work like this.	\$1,199.10 every month	\$231,677.04
Equal principal	The same principal each month ($200,000 / 360 = \$555.56$), plus interest on the balance, so the payment falls over time. Common in Europe and for business loans.	\$1,555.56 at first, falling to \$556.73	\$180,498.60
Interest-only + balloon	Pay only interest, then repay all the principal at the end (the "balloon").	\$1,000.00 a month, then \$201,000.00 at the end	\$360,000.00

Equal principal costs less interest because the balance falls faster, but the first payments are higher. Interest-only has the lowest payments and the highest total interest, plus the risk of not having \$200,000 at maturity.

The app also allows an **annuity with a balloon**: the payment is `PMT(i, n, -P, balloon)` and the balloon is paid with the last payment. Car leases and some commercial loans work like this.

Zero-interest loans (for example "0% financing") are a special case. The PMT formula would divide by zero, so the payment is simply P / n : \$12,000 over 24 months = \$500.00.

Dates. The app treats the start date as the **first payment date** and adds calendar months like Excel's `EDATE`. A loan whose first payment is on 31 January pays on 28 February, 31 March, 30 April and so on. Bi-weekly loans add 14 days. Note that 26 bi-weekly payments a year come to more than 12 monthly payments' worth. That is why "bi-weekly" plans pay off faster.

8. Investing: compound growth with contributions

Saving is a loan in reverse: you are the lender, and the balance grows. The app's default plan is **\$10,000 now plus \$500 at the end of every month, at 7% compounded monthly, for 20 years.**

In Excel: `=FV(7%/12, 240, -500, -10000) = $300,850.72`

After	Final value	You put in	Growth
1 year	\$16,919.19	\$16,000	\$919.19
5 years	\$49,972.70	\$40,000	\$9,972.70
10 years	\$106,639.02	\$70,000	\$36,639.02
20 years	\$300,850.72	\$130,000	\$170,850.72
30 years	\$691,150.47	\$190,000	\$501,150.47

From year 10 to year 20 you add \$60,000, but the balance grows by about \$194,000.

Compounding rewards time more than anything else. The \$10,000 alone, with no contributions, would grow to \$40,387.39 in 20 years.

Beginning vs end of period. If you contribute at the **beginning** of each month (an *annuity due*, type = 1), every contribution earns one more month of return. The final value becomes **\$302,370.09** instead of \$300,850.72.

Contribution step-up. Most people can save more as their salary grows. A 3% step-up means \$500 a month in year 1, \$515 in year 2, and so on. The 30-year example workbook uses this: \$10,000 + \$500/month rising 3% a year at 7% grows to **\$914,744.96**, from \$295,452.49 invested.

Goal seek answers "how much do I need?". To reach **\$1,000,000 in 30 years** at 7% with \$10,000 to start, you need **\$753.16 a month**. To reach **\$500,000 in 20 years** with \$500 a month, you need a return of **10.70%** a year. The contribution solver is exact, because the final value is a straight-line function of the contribution. The return solver uses bisection. If a target can't be reached even at 100% a year, the app says so instead of returning a made-up number. The original version did return one.

9. Inflation, real returns and tax

Inflation means future dollars buy less. To express a future amount in **today's money**, divide by $(1 + \text{inflation})^{\text{years}}$:

$$300,850.72 / 1.025^{20} = \mathbf{\$183,600.45} \text{ in today's purchasing power, at 2.5\% inflation.}$$

The **real return** is $(1 + \text{nominal}) / (1 + \text{inflation}) - 1 = 1.07 / 1.025 - 1 = \mathbf{4.39\%}$ (the Fisher equation). "Nominal minus inflation" (4.5%) is only an approximation.

Tax on gains. The app uses a simple, clearly stated assumption: **gains are taxed as they are earned**. Each month's positive growth is taxed at the chosen rate, and the tax leaves the account. A loss gives no refund. At 15%, the 20-year plan ends at **\$262,431.08** instead of \$300,850.72, with \$23,370.19 paid in tax. The tax also stops the taxed money from compounding, which is why the gap is larger than the tax itself.

In reality, many accounts tax only when you sell, or not at all (pension and tax-free accounts). Then the result lies between the taxed and untaxed figures. Say this assumption out loud in an interview.

10. NPV and IRR in plain words

A company considers a project: pay **\$100,000 today**, then receive \$25,000, \$35,000, \$40,000, \$35,000 and \$30,000 at the end of years 1–5. Its required return (hurdle rate) is **10%**.

NPV: "How much richer does this make us, in today's money?"

Discount every cash flow back to today and add them up, including the –100,000 at $t = 0$, which is not discounted:

Year	Cash flow	$\div 1.10^t$	Present value
0	–100,000	1.00000	–100,000.00
1	25,000	1.10000	22,727.27
2	35,000	1.21000	28,925.62
3	40,000	1.33100	30,052.59
4	35,000	1.46410	23,905.47
5	30,000	1.61051	18,627.64
		NPV	24,238.60

(The rounded PVs add up to 24,238.59. The exact total is 24,238.595..., which rounds to 24,238.60.)

NPV > 0 means accept: the project earns more than 10% and adds \$24,238.60 of value.

⚠ **The classic Excel trap.** Excel's `NPV(rate, values)` assumes the **first value arrives at $t = 1$** . So:

- ✓ Correct: `=-100000 + NPV(10%, 25000, 35000, 40000, 35000, 30000)` = **24,238.60**
- ✗ Wrong: `=NPV(10%, -100000, 25000, ...)` = **22,035.09**. This discounts everything by one extra year (24,238.60 / 1.1).

The app shows both, so you can see the mistake.

IRR: "What return does this project earn?"

The **IRR** is the discount rate that makes NPV exactly 0. For this project, `=IRR(B1:B6)` = **18.78%**. That is above the 10% hurdle, so accept, which agrees with NPV. The other measures in the app:

- **Profitability index** = PV of inflows / outlay = 124,238.60 / 100,000 = **1.24**.
- **Simple payback** is 3.00 years.
- **Discounted payback** is **3.77 years**: after year 3 the discounted cumulative is still –18,294.52, and year 4 brings 23,905.47, so $3 + 18,294.52 / 23,905.47 = 3.77$.

IRR edge cases, which the app handles:

- **No sign change** (all flows positive, or all negative): there is no IRR. Excel shows #NUM!, and the app shows n/a.
- **More than one sign change** can give **several IRRs**. Flows of –100, +230, –132 have IRRs of **both 10% and 20%**. The app lists every root it finds and warns you. In that case, trust NPV.

- IRR assumes cash is reinvested at the IRR itself, and it ignores project size. When two projects conflict, **NPV wins**.

11. How to use the app and the Excel file

Starting it

```
cd portfolio/finance/loan-investment-calculator
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

Step by step

1. **Loan tab.** Enter the principal, APR, term, payment frequency, first payment date and loan type. Leave compounding at "Same as payments" unless the loan states otherwise. Add a recurring extra and lump sums in the right-hand panel. Below the inputs are the KPI cards, then a green *Acceleration impact* note with the interest and time saved, four interactive charts (balance, payment composition, cumulative cash paid, calendar-year totals) and the full schedule, with Excel and CSV downloads.
2. **Investment tab.** Enter the initial amount, contribution, frequency, timing, return, compounding, years, inflation, step-up and tax. Read the caption under the inputs: it tells you the periodic rate used. The KPI cards show the final, real, contributed and growth amounts. Use the **goal-seek** box to solve for a contribution or a return.
3. **Tools tab.** - NPV/IRR evaluator: shows the $t = 0$ explanation and the wrong-Excel check. - APR → EAR table. - Two-loan comparison: rate, term and fees. It shows the difference in payment, the total cost, the break-even month for the fees, and the APR including fees.

The Excel workbooks

Open `examples/Mortgage_25yr_300k.xlsx`. It shows \$300,000 at 6% over 25 years with \$200/month extra and a \$10,000 lump sum at payment 60.

- **Blue cells on yellow are inputs (C4:C13), plus the lump-sum table (E5:F14).** Change one and everything recalculates.
- C16:C20 hold the periodic rate, the number of payments, `PMT`, the equal-principal amount and `EFFECT`.
- The summary shows a payment of \$1,932.90, 233 payments, payoff on 1 March 2046, and \$74,655.37 of interest saved against the **baseline schedule** in columns M-Q (no extras).
- Click any schedule cell: they are all formulas (`EDATE` , `ROUND` , `MIN` , `MAX` , `IF`), not typed numbers.
- The **TVM Functions** sheet has a worked example of every Excel function in this guide, including the right and wrong NPV.

`examples/Investment_Plan_30yr.xlsx` has a monthly table, an annual roll-up, and a cross-check cell that compares the table with Excel's own `FV()`.

12. Hands-on Excel exercise: build it yourself (45-60 minutes)

Do this in a blank workbook, without looking at the example file. Then compare.

Part A: loan schedule

1. Type the inputs. Format them blue, so everyone can see they are inputs:

Cell	Label	Value
B1	Principal	10000
B2	APR	12%
B3	Years	1
B4	Payments per year	12

2. Add the calculations: - B6 $=B2/B4$ gives the rate per period, 1% - B7 $=B3*B4$ gives $n = 12$ - B8 $=ROUND(PMT(B6,B7,-B1),2)$ gives **888.49**
3. Add headers in row 10: Period, Opening, Payment, Interest, Principal, Closing.
4. Fill row 11: - A11 1 - B11 $=B1$ - C11 $=IF(A11=\$B\$7, B11+D11, \$B\$8)$. The last row pays whatever is left. - D11 $=ROUND(B11*\$B\$6,2)$ - E11 $=C11-D11$ - F11 $=B11-E11$
5. Fill row 12: A12 $=A11+1$ and B12 $=F11$. Copy C11:F11 down into row 12. Then copy row 12 down to period 12.
6. **Check:** - Row 2 interest = 92.12. - The last payment = **888.47**, and the closing balance is **0.00**. - $=SUM(D11:D22)$ = **661.86**.
7. Compare with the built-in functions: $=IPMT(\$B\$6,A11,\$B\$7,-\$B\$1)$ and $=PPMT(...)$. Round them and they agree with your table, except for the cent difference in the last row (explained in section 4).
8. Now change B1 to 200000, B3 to 30, and copy the rows down to period 360. You should get 1,199.10 and total interest **231,677.04**.

Stretch goal: add an "Extra" column G with 200 in every row. Change E to $=MIN(C+G-D, B)$, so the extra is capped at the balance, and stop the rows when the balance hits 0. Total interest should fall to **151,876.18** after 252 payments.

Part B: investment table

1. Inputs: B1 initial 10000 , B2 monthly contribution 500 , B3 return 7% .
2. Headers in row 5: Year, Balance, Contributed, Growth.
3. Fill row 6: - A6 1 - B6 $=FV(\$B\$3/12, A6*12, -\$B\$2, -\$B\$1)$ - C6 $=\$B\$1+\$B\$2*12*A6$ - D6 $=B6-C6$
4. Copy down to year 20. **Check:** - Year 1 = 16,919.19 - Year 10 = 106,639.02 - Year 20 = **300,850.72**, contributed 130,000, growth 170,850.72
5. Add column E $=B6/(1+2.5\%)^A6$. Year 20 should be **183,600.45**.
6. Change the FV type argument to 1 (beginning of month). Year 20 becomes 302,370.09.
7. Bonus: in another cell type $=-100000+NPV(10\%,25000,35000,40000,35000,30000)$ and $=IRR(...)$ for the section-10 project. You should get 24,238.60 and 18.78%.

13. Presenting the project to an employer

60-second pitch

"I built a loan and investment calculator in Python and Streamlit. It produces full amortization schedules for three loan types, with extra payments, any payment frequency and correct month-end dates. It also projects investments with contributions, inflation and tax, solves for the savings you need, and evaluates projects with NPV and IRR. The part I'm proudest of is accuracy. I tested every formula against Excel's functions in LibreOffice, and found and fixed real bugs: Python's rounding being off by 3 cents on a mortgage, an IRR that crashed, a goal seek that returned nonsense. The Excel export rebuilds each schedule as live formulas, and 42,000 cells match the Python results exactly. It has 107 automated tests."

5-minute demo script

1. **(30 s)** Loan tab: \$200,000, 6%, 30 years. "The payment is \$1,199.10 and interest is \$231,677. That's more than the loan."
2. **(60 s)** Point to the composition chart. "The first payment is \$1,000 interest and only \$199 principal. Here's why." Add \$200 extra: "\$79,800 saved, 9 years earlier."
3. **(45 s)** Download the Excel file, open it, click a schedule cell to show it's a formula, and change the rate. Everything updates.
4. **(60 s)** Investment tab: \$10,000 + \$500/month at 7% for 20 years gives \$300,851. Point to the real-value card: \$183,600 in today's money at 2.5% inflation. Goal seek: "for \$1 million in 30 years I need \$753 a month."
5. **(45 s)** Tools: NPV \$24,239, IRR 18.78%. Show the wrong-NPV check: "This is a common Excel mistake, and the app explains it."
6. **(30 s)** Close with the tests and the LibreOffice verification, and the bug list in the README.

CV bullet points

- Built a Python/Streamlit loan and investment calculator (amortization, extra payments, compound growth, goal seek, NPV/IRR) whose Excel exports rebuild every schedule as live formulas; **0 mismatches across 42,000+ cells** after LibreOffice recalculation.
- Verified the TVM engine against Excel's PMT, IPMT, PPMT, FV, PV, RATE, NPER, NPV, IRR and EFFECT in 104 test cases. **Fixed 10+ defects**, including half-even vs half-up rounding, IRR overflow, and a goal seek that silently failed.
- Wrote **107 automated tests** (pytest), including regression tests for each fix, plus a beginner's guide that explains time value of money, amortization and capital budgeting.

LinkedIn post draft



How much does a \$200,000 mortgage really cost?

At 6% over 30 years, the payment is \$1,199.10, and you pay **\$231,677 in interest**. In the first month, \$1,000 of your payment is interest and only \$199 reduces the debt.

I built a Loan & Investment Calculator (Python + Streamlit) to explore questions like this: • adding \$200/month saves \$79,800 and ends the loan 9 years early • \$10,000 + \$500/month at

7% grows to \$300,851 in 20 years, of which \$170,851 is compound growth • NPV, IRR, APR vs EAR, and a classic Excel NPV mistake explained

My favourite lesson: testing the first draft against Excel exposed real bugs. One was a 3-cent rounding error, because Python and Excel round halves differently. Every Excel export is now built from live formulas and matches Python to the cent.

Code and a beginner's guide: github.com/Yasskik/portfolio

Finance #Excel #Python #Accounting #TimeValueOfMoney

14. Interview questions and sample answers

1. What is the time value of money? A dollar today is worth more than a dollar later, because it can earn interest, inflation reduces what money buys, and future payments are uncertain. To compare amounts at different dates, you move them to the same date by compounding or discounting.

2. What is the difference between APR and EAR? APR is the quoted nominal rate. EAR includes the effect of compounding within the year. 12% compounded monthly is 1% a month, which gives an EAR of 12.68%. Compare products using EAR.

3. Why is most of an early mortgage payment interest? Interest is charged on the balance, which is largest at the start. On \$200,000 at 6%, the first month's interest is \$1,000 of a \$1,199.10 payment. As the balance falls, the principal share grows.

4. How do you make an amortization schedule end at exactly zero? Round the payment and each interest amount to the cent, as a lender does. Then make the last payment equal to the remaining balance plus its interest. Otherwise rounding leaves a few cents, or a negative balance.

5. What does an extra payment do? It reduces principal directly. Every later interest charge is then smaller, so the fixed payment repays more principal and the term shortens. With \$200 a month extra on the \$200k loan, it ends in 252 payments instead of 360 and saves \$79,800.86.

6. Annuity vs equal-principal vs interest-only? - Annuity: the same payment every period. - Equal principal: the same principal each period, so the payments start higher and fall. Less total interest. - Interest-only: the lowest payments, but the full principal is due at the end, so the most interest and refinancing risk.

7. How do you handle compounding that differs from the payment frequency? Use the equivalent periodic rate, $i = (1 + \text{APR}/m)^{(m/p)} - 1$, so the effective annual rate is preserved. For a Canadian 6% mortgage compounded semi-annually and paid monthly, that is 0.49386% a month, not 0.5%.

8. Ordinary annuity vs annuity due? In an ordinary annuity, payments are at the end of each period. In an annuity due, they are at the beginning, so each payment earns one more period of

interest. The FV of an annuity due = FV of an ordinary annuity $\times (1 + i)$. In Excel, it's the `type` argument.

9. What is the real return? The return after inflation: $(1 + \text{nominal}) / (1 + \text{inflation}) - 1$. At 7% and 2.5%, that is 4.39%.

10. Explain NPV and the Excel NPV trap. NPV is the sum of all cash flows discounted to today. Positive means the project beats the required return. Excel's `NPV()` assumes the first value is one period away, so you must add the $t = 0$ outlay outside the function: `=CF0 + NPV(r, CF1:CFn)`.

11. When can IRR mislead you? - When the cash flows change sign more than once: there can be several IRRs ($-100, +230, -132$ has IRRs of 10% and 20%). - When there is no sign change: there is no IRR. - When comparing projects of different sizes. - It also assumes reinvestment at the IRR.

NPV is the more reliable decision rule.

12. How did you verify the calculations? - I compared every TVM function with LibreOffice's built-in Excel functions in 104 cases. - The Excel export is recalculated by headless LibreOffice and compared cell by cell with Python across 15 scenarios: 42,256 cells, 0 mismatches. - A deliberately tampered workbook is caught, so I know the check works. - 107 pytest tests run all of this automatically.

Bonus: what bugs did you find? - Python rounds half to even, but Excel rounds half up. That gave a 3-cent interest difference on a mortgage. - `RATE` returned -164% when there was no solution. - `IRR` crashed on large losses. - The goal seek returned 200% for unreachable targets. - The Excel schedule never reached a zero balance.

How to show you really know the project

- Be able to do the 12-month schedule in section 4 **by hand**, and build section 12 in Excel from scratch, without help.
- Explain any one row of the mortgage workbook: where each number comes from.
- Know the conventions table in the README: periodic rate, rounding, timing, tax assumption, NPV at $t = 0$.

Improvements you can make yourself (pick 2-3)

1. **Rate changes:** let the rate change at a given period, as in an adjustable-rate mortgage, and recalculate the payment from that point.
2. **Recast option:** after a lump sum, keep the same end date and lower the payment, instead of shortening the term.
3. **Fees and true APR:** add an origination fee to the loan tab and use `RATE` to compute the APR including fees (the comparison tab already does this; reuse it).
4. **Day-count interest:** charge interest as $\text{balance} \times \text{APR} \times \text{days} / 365$ using the actual dates, and compare with the monthly method.
5. **XNPV / XIRR:** let the tools tab accept cash flows on irregular dates.
6. **Rent vs buy:** compare buying with a mortgage against renting and investing the difference.
7. **Taxes on withdrawal:** add a "taxed only at the end" option to the investment tab, and show the difference from the "taxed as earned" assumption.

For each one, write the formula on paper first, add a test with a number you computed in Excel, and only then write code.

15. Your study plan for the next 7 days

Day	Goal (about 1-1.5 hours)	Check yourself
1	Sections 1-3. Calculate FV, PV and EAR on paper.	$100 \times 1.05^{10} = 162.89$; EFFECT(12%,12) = 12.68%
2	Section 4. Do the 12-month schedule by hand for rows 1-3, then in Excel.	Payment 888.49, last payment 888.47, interest 661.86
3	Sections 5-7. Use the app's loan tab: try different extras and loan types.	\$200 extra saves \$79,800.86; equal-principal interest \$180,498.60
4	Section 12, Part A, full 360-row mortgage in Excel, with the stretch goal.	231,677.04 and 151,876.18
5	Sections 8-9 and Part B. Use the investment tab and goal seek.	300,850.72; real 183,600.45; \$753.16/month for \$1M
6	Section 10. Calculate NPV by hand, then show the Excel NPV trap.	24,238.60 vs 22,035.09; IRR 18.78%
7	Sections 13-14. Rehearse the pitch and the demo out loud, and answer the 12 questions without notes. Start one improvement from section 14.	A friend can follow your demo in 5 minutes

Good luck. Once you can explain why the first mortgage payment is \$1,000 of interest, you understand more about loans than most borrowers ever do.